

11-agreement.pdf

phase commit - 2PARTICIPANTES - 1 decion - coordinator wait for decion

Participant failure 1PARTICIPANTE - coordinator don't wait and take a decion alertando todos os participantes

coordinator failure - o coordenador falha e os participantes bloqueiam

Resumo:

Tolerância a Falhas:

Os sistemas distribuídos devem lidar com falhas para garantir que continuem a funcionar corretamente.

Existem dois principais modelos de falhas:

- Omissivas: Quando algo é "omitido", como um processo que para de funcionar (crash) ou uma mensagem que se perde na comunicação.
- Assertivas (Byzantinas): Quando há erros mais complexos, como mensagens corrompidas ou deliberadamente erradas.

A capacidade de um sistema lidar com essas falhas evita falhas catastróficas que poderiam comprometer todo o sistema.

Transações Distribuídas (2PC):

O protocolo 2-phase commit (2PC) é usado para coordenar transações entre vários participantes de um sistema distribuído.

- Fase 1 - Prepare: O coordenador pergunta aos participantes se estão prontos para realizar a transação.
- Fase 2 - Commit/Rollback: Após as respostas, o coordenador decide concluir (commit) ou desfazer (rollback) a transação.

O 2PC é vulnerável a falhas:

- Se um participante falhar, o sistema pode precisar desfazer a transação.
- Se o coordenador falhar, os participantes podem ficar "bloqueados", aguardando uma decisão.

Replicação:

A replicação mantém múltiplas cópias de dados em diferentes nós para escalabilidade e tolerância a falhas. Isso garante que o sistema continue funcionando mesmo que alguns nós falhem.

- Quorums: Para garantir consistência, operações de leitura e escrita devem aceder a um número mínimo de réplicas (o quorum).
- Se $N=5$ réplicas, pode-se configurar $NW=3$ e $NR=3$ para garantir que leituras e escritas acessem as mesmas réplicas.

Consenso:

O problema de consenso ocorre quando processos distribuídos precisam concordar numa única decisão, mesmo com falhas. Por exemplo, no algoritmo Paxos, participantes propõem valores, e o sistema deve escolher apenas um.

- É essencial para sistemas como Kubernetes (gerir containers) ou blockchains (gerir transações).
- Consenso garante consistência, mesmo em situações de falhas, mas pode ser complexo e lento.

Escalabilidade e Eficiência

Sistemas distribuídos devem crescer de forma eficiente para suportar mais usuários ou dados sem perder desempenho.

- Estratégias eficientes: Replicação, quorum, e consenso ajudam a gerenciar a carga de trabalho enquanto toleram falhas.
- Um bom sistema balanceia escalabilidade e tolerância a falhas, mantendo o desempenho mesmo em grande escala.

Bancos de dados distribuídos como o Amazon Aurora implementam replicação e quorums para alcançar esse equilíbrio.

Perguntas:

1. Necessidade de migração de código cliente para o servidor e modelo de threading adequado para o servido.

R: Migrar o código para o servidor melhora o desempenho, segurança e consistência, reduzindo a carga no cliente. Essa abordagem centraliza a lógica

crítica, facilitando sincronização e controlo de dados. O modelo de threading adequado utiliza filas para gerir requisições de forma eficiente, reaproveitando threads e evitando sobrecarga. Isto aumenta a escalabilidade do servidor, permitindo lidar com múltiplas conexões. O uso de filas organiza as requisições, priorizando o processamento ordenado e controlado.

2. Explique o que é Quorum tolerante a falhas em sistemas distribuídos.

Um quorum é o número mínimo de réplicas necessárias para realizar operações de leitura ou escrita em sistemas distribuídos. Ele garante consistência e disponibilidade, mesmo com falhas, seguindo regras como $NR + NW > N$ e $NW > N/2$. Essas regras evitam conflitos e asseguram acesso aos dados mais recentes. Em quorums tolerantes a falhas, leituras e escritas são projetadas para continuar funcionais mesmo com falhas parciais.

- $N=5$ e $f=1$, configuram-se $NR=3$ e $NW=3$ para alta resiliência.

3. Qual é estratégia partilhada pela disseminação epidêmica e pelo Chord DHT para eficiência em grande escala? Justifique

A disseminação epidêmica propaga dados aleatoriamente entre nós, imitando a propagação de doenças, enquanto o Chord usa hashing distribuído para localizar dados em tempo logarítmico. Ambos limitam interações a um subconjunto de nós, reduzindo custos de comunicação e processamento. Essa abordagem promove escalabilidade e robustez, permitindo operação eficiente mesmo em redes muito grandes. A combinação de simplicidade e eficiência garante que os sistemas distribuídos sejam robustos a falhas e dinâmicos.